

PHP y sus frameworks

Tecnologías para el desarrollo web en entorno servidor

Laravel · Symfony

Desarrollo Web en Entorno Servidor — Leandro Tristá Hernández
Manuel Rey Laguarde

Índice

1. Lenguaje de programación
2. Requisitos para el desarrollo
3. Requisitos para la ejecución de las páginas
4. Compatibilidad con servidores web
5. IDEs y herramientas de desarrollo
6. Ejemplos de páginas y programas
7. Sitios web públicos que usan PHP
8. Comparativa con otras tecnologías
9. Prueba de concepto

1. Lenguaje de programación: PHP

- PHP (PHP: Hypertext Preprocessor)
- Lenguaje de programación interpretado del lado del servidor que genera contenido web dinámico
- Se ejecuta en el servidor, no en el navegador.
- Genera HTML, JSON u otros formatos.
- Permite crear páginas dinámicas conectadas a bases de datos.

2. Requisitos para el desarrollo

- Editor o IDE con soporte para PHP (PHPStorm, VS Code...)
- Intérprete de PHP instalado (php -v para comprobar)
- Gestor de paquetes Composer para librerías.
- Docker o XAMPP para replicar/estandarizar los entornos de desarrollo.

3. Requisitos para la ejecución de las páginas

- Servidor web que reciba la petición HTTP (Apache, Nginx...)
- Intérprete de PHP instalado en el servidor (Zend Engine)
- Los ficheros deben tener extensión .php
- El código PHP va dentro de `<?php ... ?>` y se ejecuta en el servidor
- El cliente solo recibe el HTML resultante (nunca el código fuente)
- Opcionalmente una base de datos (MySQL, MariaDB, PostgreSQL).

4. Compatibilidad con servidores web

PHP es multiplataforma y compatible con los principales servidores web

- Apache (históricamente el más utilizado).
- Nginx (muy común actualmente).
- IIS (Internet Information Services) en Windows.
- Integración mediante módulos `mod_php` o gestor de procesos PHP-FPM (PHP FastCGI Process Manager).

5. IDEs y herramientas usadas para desarrollar

PHPStorm (JetBrains) — IDE profesional con opción de pago, muy completo

Visual Studio Code — gratuito, con extensiones PHP Intelephense

NetBeans y Eclipse PDT — IDEs gratuitos clásicos

Sublime Text / Vim / Neovim — editores ligeros

Herramientas complementarias:

- Composer — gestor de dependencias

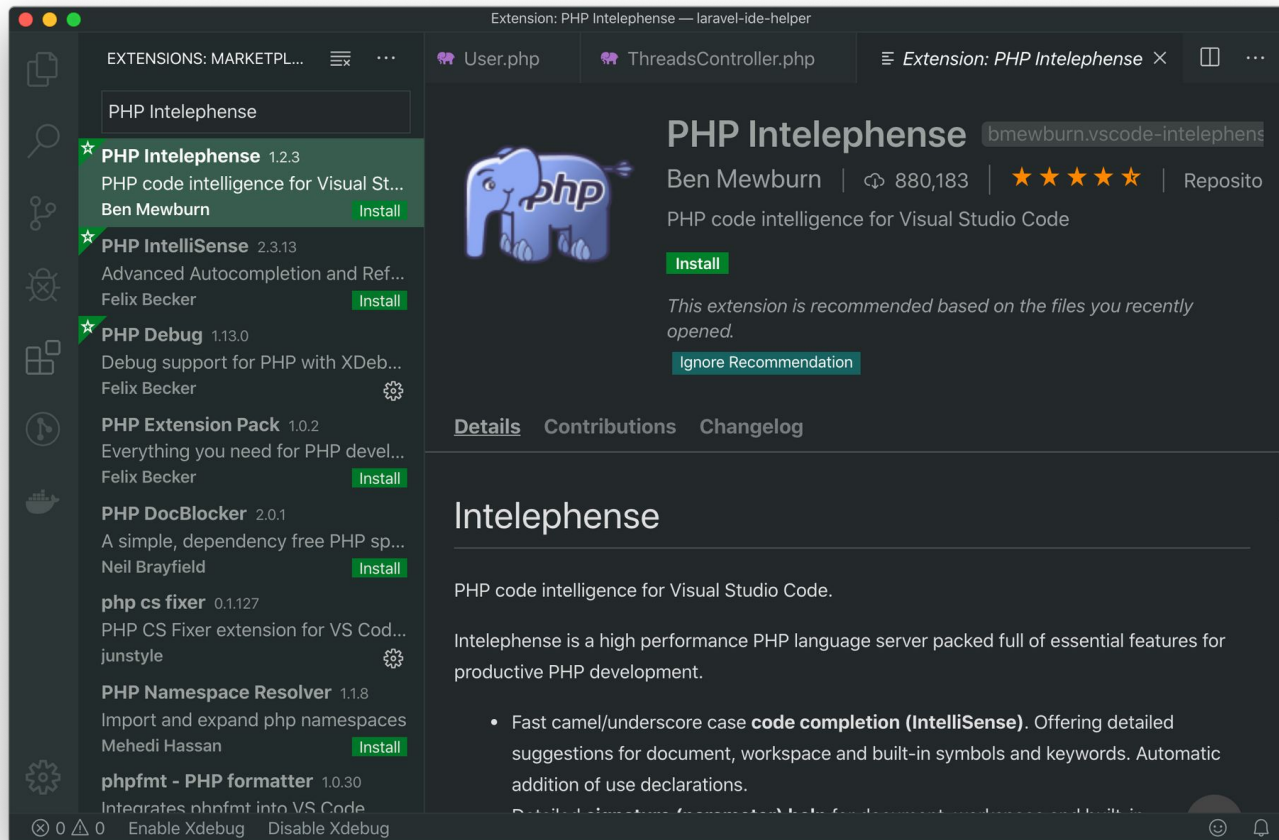
- PHPUnit / Pest — pruebas unitarias

- Xdebug — depuración y perfiles

- Laravel Herd / Sail — entornos de desarrollo

IDE en la práctica: Visual Studio Code

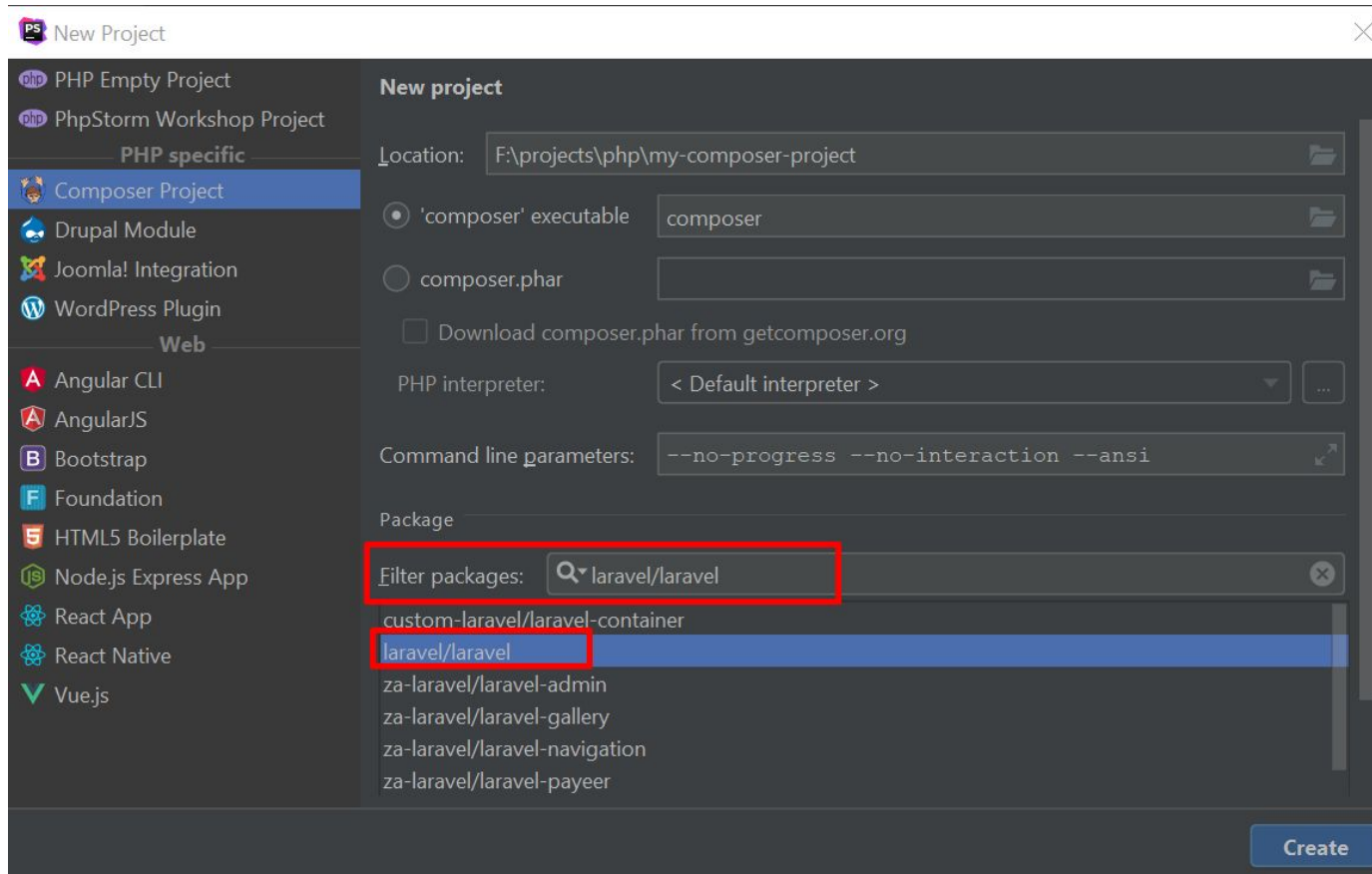
Ejemplo real: extensiones instaladas en VS Code para trabajar con PHP



- **PHP Intelephense**
autocompletado, definiciones y refactorización
- **PHP Debug**
depuración paso a paso con Xdebug
- **PHP DocBlocker**
generación automática de DocBlocks
- **PHP Namespace Resolver**
importar clases y expandir namespaces
- **Laravel Extra Intellisense / Blade**
soporte específico para Laravel

IDE en la práctica: PhpStorm + Laravel

Ejemplo real: creación de un proyecto Laravel desde el asistente de PhpStorm



- PhpStorm trae soporte nativo para PHP, Composer y Laravel
- **File** → **New Project** → **Composer Project**
- En "Package" se **filtra** laravel/laravel
- El IDE descarga el esqueleto del framework automáticamente
- Incluye terminal integrada, depurador Xdebug y asistencia para artisan

7. 6 grandes aplicaciones web basadas en PHP:

1. Facebook



2. Wikipedia



3. Yahoo



4. AbsoluteWrite



5. Tumblr



WordPress



8. Comparativa con otras tecnologías

vs Node.js

JavaScript Asíncrono / Event-Loop

Arquitectura de Memoria

PHP utiliza modelo 'Shared-Nothing' (aislamiento total por petición). Node.js comparte un hilo único con bucle de eventos.

Despliegue y Mantenimiento

PHP se integra directo en Apache/Nginx con FPM. Node requiere gestores de procesos persistentes (PM2, Docker, systemd).

¿Dónde gana PHP?

Imposible tumbar el servidor por un error no capturado; coste de infraestructura y hosting drásticamente menor.

vs Java (J2EE / Spring)

Compilado / Empresarial

Ciclo de Desarrollo

PHP es interpretado: guardar y refrescar en navegador. Java exige compilación, empaquetado (JAR/WAR) y arranque lento.

Consumo de Recursos

PHP arranca y libera memoria en cada petición HTTP. Las máquinas virtuales Java (JVM) exigen reservas altas de RAM.

¿Dónde gana PHP?

Velocidad de entrega (Time-to-Market), agilidad para cambios rápidos y menor complejidad en aplicaciones web comunes.

vs Python (Django / Flask)

Propósito General / WSGI

Naturaleza del Lenguaje

PHP fue concebido nativamente para la web (\$_GET, \$_POST). Python requiere capas intermedias de adaptación (WSGI/ASGI).

Rendimiento HTTP Concurrente

Con PHP 8, OPcache y compilación JIT, PHP supera habitualmente a Python en peticiones HTTP dinámicas.

¿Dónde gana PHP?

Hosting universal inmediato sin configuración de servidores de aplicación (Gunicorn) y mayor cuota en la web real.

9A. Prueba de Concepto: Inicialización y Código (1/2)

Construcción ágil de un microservicio desde cero.

PASO 1 | Creación del Proyecto

```
composer create-project  
laravel/laravel micro-demo  
cd micro-demo
```

- **Descarga oficial:** Composer descarga el esqueleto base limpio y sus dependencias en vendor/.
- **Auto-configuración:** Genera de forma automática el archivo de entorno .env y la clave APP_KEY.
- **Entorno ligero:** No requiere configurar motores externos de bases de datos ni servicios de fondo.

PASO 2 | Ruta con Array Asociativo

```
// routes/web.php  
Route::get('/', function () {  
    return [  
        'estado' => 'Servidor activo',  
        'tecnologia' => 'Laravel ' .  
app()->version(),  
        'php' => phpversion()  
    ];  
});
```

- **Edición de un solo archivo:** Modificación directa en routes/web.php sustituyendo la vista welcome original.
- **Retorno estructurado:** Se define un array asociativo con datos de diagnóstico y versiones en runtime.
- **Serialización nativa:** Laravel detecta el array y lo transforma internamente en una estructura JSON válida.

9B. Prueba de Concepto: Servidor

Respuesta HTTP inmediata y transformación transparente del entorno servidor

PASO 3 | Ejecución del Servidor Integrado

```
php artisan serve
```

```
# Salida consola:
```

```
# INFO Server running on  
[http://127.0.0.1:8000]
```

- **Servidor CLI:** Arranca el servidor web integrado de PHP sin necesidad de configurar Apache ni Nginx.
- **Enlace local:** Mapea la petición entrante al puerto 8000 apuntando directamente a public/index.php.
- **Entorno de desarrollo:** Permite verificar endpoints de forma inmediata sin alterar la configuración del sistema.

RESULTADO | Content Negotiation Automática

```
// Cabeceras HTTP generadas:  
HTTP/1.1 200 OK  
Content-Type: application/json
```

```
// Respuesta en navegador:
```

```
{  
  "estado": "Servidor activo",  
  "tecnologia": "Laravel 11.x",  
  "php": "8.3.6"  
}
```

- **Inyección de cabeceras:** El pipeline añade automáticamente Content-Type: application/json al detectar el array.
- **Sin json_encode manual:** El framework gestiona la serialización y el envío del payload en formato nativo REST.
- **Tiempo récord:** Microservicio funcional listo para consumir por clientes web o móviles en menos de 2 minutos.

9B. Prueba de Concepto: Vizualizacion Pagina Funcional

 http://localhost:8000

JSON

Datos sin procesar

Cabeceras

Guardar

Copiar

Contraer todo

Expandir todo

 Filtrar JSON

estado:

"Servidor activo"

tecnologia:

"Laravel 13.32.0"

php:

"8.3.6"

10. Conclusiones

Síntesis de los conceptos clave abordados durante la presentación

PUNTO 01

Lenguaje e IDEs

Modernidad y Entorno

- **PHP 8.x y POO:** Supera el código 'legacy' incorporando tipado estricto opcional, atributos y compilador JIT.
- **Ecosistema de IDEs:** Pleno soporte profesional: extensiones en VS Code (Intelephense, Blade) y asistentes nativos en PhpStorm.

PUNTO 02

Servidores y Despliegue

Arquitectura y Ejecución

- **Compatibilidad Universal:** Ejecución flexible sobre Apache (mod_php/FPM), Nginx y servidores integrados ('php artisan serve').
- **Aislamiento de Memoria:** Modelo 'Shared-Nothing' por petición HTTP, evitando caídas globales ante fallos puntuales (frente a Node.js).

PUNTO 03

Frameworks y Demostración

Laravel y Presencia Real

- **Productividad (PoC):** Creación ágil de microservicios: enrutamiento limpio y serialización automática a JSON sin capas complejas.
- **Web en Producción:** Sustenta más del 40% de internet mediante WordPress, WooCommerce y plataformas masivas como Wikipedia.